Project 1 Report: A Simple Calculator

1. Project Overview

This project implements a command-line calculator in pure C. The whole program is written in one source file, `calculator.c`, and can be compiled by `gcc`. The calculator supports basic arithmetic operations, several extra functions, interactive input, and large-number decimal processing.

The main goal of this project was not only to finish the minimum requirements, but also to design a calculator with clearer structure, better extensibility, and more reliable handling of numbers than ordinary built-in types such as `int`, `long long`, `float`, or `double`.

2. Implemented Features

The current calculator supports the following functions:

- Addition: `+`
- Subtraction: `-`
- Multiplication: `*` and `x`
- Division: `/`
- Square root: `sqrt`
- Integer power: `pow`

It also supports:

- Command-line mode
- Interactive mode
- `help`, `quit`, and `exit` commands
- Scientific notation input such as `1e3`, `2.5E-4`
- Error messages for invalid input and division by zero

3. Program Interface

3.1 Command-Line Mode

Examples:

```
./calculator 12.5 + 3.75
./calculator 2e3 '*' 4
./calculator sqrt 2
./calculator pow 2 10
./calculator --help
```

3.2 Interactive Mode

When the program is started without arguments, it enters interactive mode.

Example session:

```
12.5 + 3.75
sqrt 2
pow 2 10
help
quit
```

4. Core Design

4.1 BigDecimal Structure

The core data structure of this project is `BigDecimal`:

```
typedef struct {
    int sign;
    int scale;
    int len;
    int digits[MAX_DIGITS];
} BigDecimal;
```

Its purpose is to represent signed decimal numbers with arbitrary precision inside a fixed-size digit array.

Meaning of each field:

- `sign`: sign of the number, positive, negative, or zero
- `scale`: number of digits after the decimal point
- `len`: total number of stored digits
- `digits[]`: digits stored in reverse order for easier arithmetic

For example, the number `123.45` is stored as digits `5 4 3 2 1`, with `scale = 2`.

This design makes addition, subtraction, multiplication, and normalization easier to implement.

4.2 Input Parsing

The function `read_number` is responsible for reading a string and converting it into a `BigDecimal`. It supports:

- optional sign
- decimal point
- scientific notation using `e` or `E`

This allows the program to handle input such as:

- `123`
- `-0.25`
- `1e6`
- `-3.2E-5`

4.3 Output Formatting

The function `write_number` converts a `BigDecimal` back into a readable string. According to the size of `scale`, it prints the number either:

- in normal decimal form
- or in scientific notation if the decimal point would become too inconvenient

This keeps the output readable even when the result is extremely large or extremely small.

5. Arithmetic Implementation

5.1 Addition and Subtraction

For addition and subtraction, the program first aligns the decimal point of two numbers by comparing their `scale`. Then it performs digit-by-digit arithmetic with carry or borrow.

This is similar to manual decimal arithmetic and avoids the precision loss of floating-point numbers.

5.2 Multiplication

Multiplication is implemented using the classical grade-school method:

- multiply each digit pair
- accumulate partial products
- propagate carry

The decimal scale of the result is the sum of the scales of the two operands.

5.3 Division

Division was the most difficult part of the project.

The current version performs division by repeatedly determining one quotient digit at a time, which is close to the idea of long division. It also keeps a configurable number of valid digits and then rounds the result.

To avoid returning too many digits, I added a helper function `save_lens` to keep the result within the required precision. At the same time, if the operation ends exactly, the program returns the exact value instead of forcing an unnecessary modification.

5.4 Square Root

The `sqrt` function is implemented using fixed-iteration binary search.

Given a non-negative number `x`, the algorithm:

1. sets the search interval to `[0, max(1, x)]`
2. repeatedly computes the midpoint
3. compares `mid * mid` with `x`
4. keeps the correct half-interval

After a fixed number of iterations, the left boundary is used as the approximation result.

This implementation is simple, stable, and easy to explain in an interview.

5.5 Power

The `pow` function is implemented using fast exponentiation for non-negative integer exponents.

Instead of multiplying the base repeatedly `n` times, the program uses exponentiation by squaring, which greatly reduces the number of multiplication steps.

This is much more efficient than a naive loop when the exponent is large.

6. Error Handling

The program checks several types of invalid input:

- invalid number format
- unsupported operator
- division by zero
- negative input for `sqrt`
- non-integer exponent for `pow`
- too large digit count
- too large exponent in scientific notation

This part was important because a calculator should not simply crash or output meaningless values when input is invalid.

7. Code Structure and Style

I paid attention to code organization and tried to keep each function focused on a single task.

Examples:

- `read_number` and `write_number` handle conversion between strings and internal numbers
- `compare_abs`, `add_abs`, and `subtract_abs` implement basic decimal operations
- `add_numbers`, `subtract_numbers`, `multiply_numbers`, and `divide_numbers` form the arithmetic layer
- `execute_binary`, `execute_sqrt`, and `execute_pow` connect arithmetic logic with user input
- `run_interactive` manages the REPL-like interaction mode

I also kept several debug helper functions inside `#ifdef DEBUG` blocks so they can be enabled for testing without affecting normal builds.

8. Testing

I tested the program with:

- ordinary integers
- decimals
- scientific notation
- negative numbers
- large numbers
- edge cases such as zero and invalid input

Representative examples:

```
12.5 + 3.75 = 16.25
3 / 2 = 1.5
2 / 3 = 0.66666667
sqrt(2) = 1.41421356...
pow(2, 10) = 1024
```

Examples of invalid input handling:

```
1 / 0
sqrt -1
pow 2 1.5
abc + 1
```

9. Difficulties and Solutions

The hardest part of the project was decimal division and precision control.

Built-in floating-point numbers would have made the implementation shorter, but they would also introduce binary floating-point error. Therefore, I chose to implement my own decimal structure and digit-based arithmetic.

Another difficulty was supporting both normal decimal input and scientific notation while keeping the parser simple. This was solved by converting the exponent into changes of `scale`.

For `sqrt`, I chose binary search instead of more complicated methods such as Newton iteration, because it is easier to verify and explain.

10. Highlights of My Work

The main highlights of this project are:

- pure C implementation in a single source file
- custom `BigDecimal` representation instead of built-in floating-point numbers
- support for scientific notation
- support for large numbers beyond ordinary integer range
- interactive mode and command-line mode
- additional functions `sqrt` and `pow`
- readable modular structure

11. Possible Future Improvements

If I continue to improve this calculator, I would consider:

- making division precision control more flexible
- improving the parser so expressions without spaces are handled more naturally
- supporting more functions such as `sin`, `cos`, `log`, or parentheses
- adding automated unit tests
- separating the program into multiple modules if the project becomes larger

12. Conclusion

This project successfully implements a usable calculator in C and goes beyond the minimum requirements. It supports exact decimal-style processing through a custom data structure, handles interactive and command-line input, and includes several additional mathematical functions.

More importantly, the project helped me practice:

- C programming
- command-line development
- custom numeric representation
- algorithm design for arithmetic operations
- program organization and debugging

Overall, this calculator is a solid and extensible foundation for a larger numerical tool.